

Python Cheatsheet

State: Totally unfinished! I publish it to be hopefully still helpfull.

This Cheatsheets have been heavily influenced by the Cheatsheets of labex.io (much Copy&Paste) which are licensed under the MIT License, specifically: Python Numpy Pandas matplotlib

Python

Tbd

For people from other languages:

- no “;” (each statement needs its own line)
- no { } brackets; blocks are defined by intention. if, for, while have a : at the end. e.g.

```
if a == 2:
    do_if_a_is_two()
do_either_way()
```

You can choose the intention style&amount but you need to be consistent for each intention-level.

- Type definitions are optional.
- The first parameter of methods is self and is not explicitly passed when the method is called.

NumPy

Mathematics-library. Often much faster&better than Python Maths.

```
import numpy as np
```

Creating:

```
new_list = np.array([1, 2, 3, 4])
new_matrix = np.array([[1, 2], [3, 4]])
#...
```

```
np.zeros(3) # np.array([0.0, 0.0, 0.0])
# notice that this are floating point numbers
np.ones((2,3), dtype=int)
# np.array([[1, 1, 1], [1, 1, 1]])
np.full((2, 3), 7)
# np.array([[7, 7, 7], [7, 7, 7]])
```

```
np.arange(6) #np.array([1, 2, 3, 4, 5]) - excluding 6; as range(6) also does
np.arange(2, 6) # np.array([2, 3, 4, 5]) - including 2 but excluding 6 as range(2,6) also does
```

```
np.linspace(0, 1, 5) # 5 values from 0 to 1 -
np.array([0., 0.25, 0.5, 0.75, 1.])
```

```
# Random values between 0 and 1 - in a 2x3 matrix
np.random.random((2, 3))
# Random integers from 1-9 in a 3x3 matrix
np.random.randint(0, 10, size=(3, 3))
# Normal distribution - mean 0, standard deviation 1, list of 5 random numbers
np.random.normal(0, 1, size=5)
# Set random seed for reproducibility
np.random.seed(42)
```

Functions:

```
np.sin(value) #as radiant
np.cos(value) #as radiant
np.tan(value) #as radiant
np.exp(value)
np.log(value)
np.log10(value)
np.sqrt(value)
np.power(value, 3)
```

Aggregate functions:

```
# Basic statistics
np.sum(arr)
np.mean(arr)
np.std(arr) # Standard deviation
np.var(arr) # Variance
np.min(arr)
np.max(arr)
# Along specific axis
np.sum(arr2d, axis=0) # Sum along rows
np.mean(arr2d, axis=1) # Mean along columns
```

Accessing Data:

```
# Single dimension
arr[0] # First element
arr[-1] # Last element
# 2D array
arr2d[0, 1] # Row 0, Column 1
arr2d[1] # Entire row 1
# Slicing
arr[1:4] # Elements 1 to 3
arr[:,2] # Every second element
arr[::-1] # Reverse array
```

get a filtered slice:

```
arr[arr > 5] #Only values > 5
arr[(arr > 2) & (arr < 8)]
arr[(arr < 2) | (arr > 8)]
```

TODO: Everything from where Manipulation:

```
# Element-wise operations
arr1 + arr2
arr1 - arr2
arr1 * arr2
arr1 / arr2
arr1 ** 2 # Squaring
arr1 % 3 # Modulo operation
np.where(arr > 5, 'high', 'low')
```

Convert types:

```
arr.astype(type)
#type can be a python-type like float, int, str or
a numpy type like np.float32, np.float64,
np.int16, ...
```

Properties:

```
arr.shape # Array dimensions
arr.size # Total number of elements
arr.ndim # Number of dimensions
arr.dtype # Data type of elements
arr.itemsize # Size of each element in bytes
arr.nbytes # Memory usage in bytes
```

Pandas

```
import numpy as np
```

Filtering Stuff

```
df[df["age"] > 50]
```

Accessing Stuff:

```
#Get specific column  
df["col_abc"] # or df[["col_abc", "col_def"]]  
#Get a specific row by index  
df.loc[3:100] # NumPy compared to normal Python and  
NumPy includes the 100!  
#Get the i'th row (independent of index-value)  
df.iloc[3]
```